ADR 003 - Broker topic versioning concept

Status	PROPOSED
Date	Jan 14, 2026
Deciders	@David Weiss, @Blumenstock, Werner (DI FA DSP SEN TRS), @Thomas Weinschenk, @Konrad Heidrich, @Christian Franz
Consulted	Michael Heller
Informed	

Context and Problem Statement

The upcoming changes to the messaging guideline require introducing breaking, backward-incompatible modifications to the existing message formats. Currently, there is no mechanism in place—neither at the topic level nor within the message payload itself—that allows distinguishing between different message versions.

This lack of versioning leads to ambiguity and incompatibility, especially when both old and new clients are expected to coexist and communicate reliably. To ensure backward compatibility and a clear migration path, a versioning strategy must be introduced at the broker level.

There are multiple technical approaches to implementing versioning in MQTT, each with its own advantages and trade-offs. The goal is to evaluate these options and select a strategy that balances clarity, compatibility, and maintainability across MQTT 3.1.1 and MQTT 5 environments.

Decision Drivers

- Backward compatibility with existing clients
- Clear and maintainable version separation
- Compatibility with MQTT 3.1.1 (required) and MQTT 5 (optional enhancements)
- Low complexity for client implementations
- Transparent deprecation of older versions
- Support for parallel operation of different API generations

Considered Option

Topic Versioning

Either: Use versioned topic prefixes (e.g., `Oi4/v1/<ServiceType>/<AppId>/...`, `Oi4/v2/<ServiceType>/<AppId>/...`).

Or: Use versioned as part of the Oi4 prefix (e.g., `Oi4v1/<ServiceType>/<AppId>/...`, `Oi4v2/<ServiceType>/<AppId>/...`). This is similar to Sparkplug 6.1.1. namespace Elements (e.g. spAv1.0, spBv1.0)

Pros:

- ✅ Clear and unambiguous separation of protocol versions
- ✅ Clients can selectively subscribe based on supported versions
- ✅ Easy to deprecate old versions
- ✅ Fully supported in MQTT 3.1.1 and MQTT 5
- ✅ Simple broker configuration

Cons:

- ❌ Increased topic complexity
- ❌ Possible duplication of unchanged messages across versions
- ❌ Slightly more overhead when rolling out a new version

Decision Outcome/ Chosen Option

All MQTT topics will be versioned using a topic prefix corresponding to the API generation. Even if specific message types have not changed between versions, they will still be included under the new version prefix to ensure consistency and simplicity.

We decided, similar to sparkplug, to add versioning information to our namespace identifier (`Oi4` ⇒ `Oi4v2`).

Consequences/ Pros and Cons

✅ Clear and consistent topic hierarchy across all message types

✅ Easier onboarding and understanding for new developers

✅ Simplifies client-side logic (subscribe to one version only)

✅ Enables clean cut-over and controlled deprecation strategy

✅ Supports both MQTT 3.1.1 and MQTT 5 without relying on advanced features

❌ More topics in use (including duplication of unchanged messages)

❌ Migration of all message producers and consumers required per version

❌ Slightly higher maintenance burden for publishing infrastructure

More Information

- MQTT 3.1.1 Specification: MQTT Version 3.1.1
- MQTT 5 Specification: MQTT Version 5.0